# GenAI. LaTable. Diffusion speedup

A. Ryzhikov[1], S. Ali[1], A. Akhmatova[1], T. Ramazyan[1], K. Aithadzhaev[1],A. Khizhik[1]

[1]National Research University Higher School of Economics


LAMBDA • HSE

April 1, 2025

LaTable

# Introduction

**Motivation:**

- Tabular data is widely used but lacks strong foundation models.
- Current models fail to generalize across datasets due to heterogeneous structures.
- LaTable is a generative diffusion model designed for tabular data.

# Challenges in Tabular Generation

- Different datasets contain varying features and structures.
- No natural order of columns (permutation invariance needed).
- Combination of categorical and numerical data complicates modeling.

# LaTable: Model Overview

**Key Components:**

- Uses a diffusion model to iteratively refine noisy tabular data.
- Employs a transformer encoder for feature interaction.
- Integrates metadata and categorical embeddings using a pretrained LLM.

# Mathematical Formulation: Diffusion Process

**Forward Process:**

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I) \tag{1}$$

**Reverse Process:**

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \tag{2}$$

# Feature Representation and Transformer Encoder

**Feature Embeddings:**

$$h_j = g_{\text{ic}}(x_j^t) + g_r(r) + g_s(s_j) + g_t(t) \tag{3}$$

- $h_j$: Final embedded representation of feature $j$.
- $x_j^t$: Noised feature value at timestep $t$.
- $g_{\textbf{ic}}(x_j^t)$: Maps noised feature value into hidden space.
- $g_r(r)$: Maps dataset description into hidden space.
- $g_s(s_j)$: Maps feature name into hidden space.
- $g_t(t)$: Maps diffusion timestep into hidden space.

**Numerical Features:**

$$\hat{x}_j = g_{\text{on}}(h_{\text{out}}) \tag{4}$$

**Categorical Features:**

$$p(c_i) = \text{softmax}(\, WC^T\hat{c}) \tag{5}$$

# Training and Loss Functions

**Combined Loss Function:**

$$L(x, \hat{x}) = \frac{1}{d_k} \left( \sum_{j \in I_{\text{cat}}} \text{CE}(x_j, \hat{x}_j) + \sum_{j \notin I_{\text{cat}}} \text{MSE}(x_j, \hat{x}_j) \right) \tag{6}$$

# Conclusion

**Key Takeaways:**

- LaTable is a pioneering diffusion model for tabular data.
- Transformer-based architecture enables feature interaction.
- Future improvements will focus on scaling and bias reduction.

# Advanced diffusion

# DALL-E

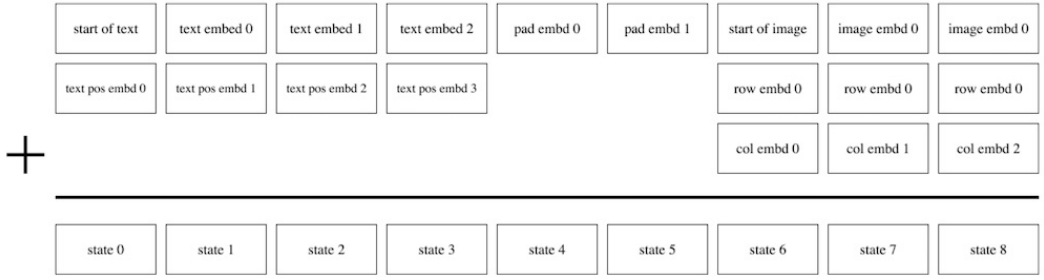



*Figure 10.* Illustration of the embedding scheme for a hypothetical version of our transformer with a maximum text length of 6 tokens. Each box denotes a vector of size $d_{\text{model}} = 3968$. In this illustration, the caption has a length of 4 tokens, so 2 padding tokens are used (as described in Section 2.2). Each image vocabulary embedding is summed with a row and column embedding.

# Background. CLIP

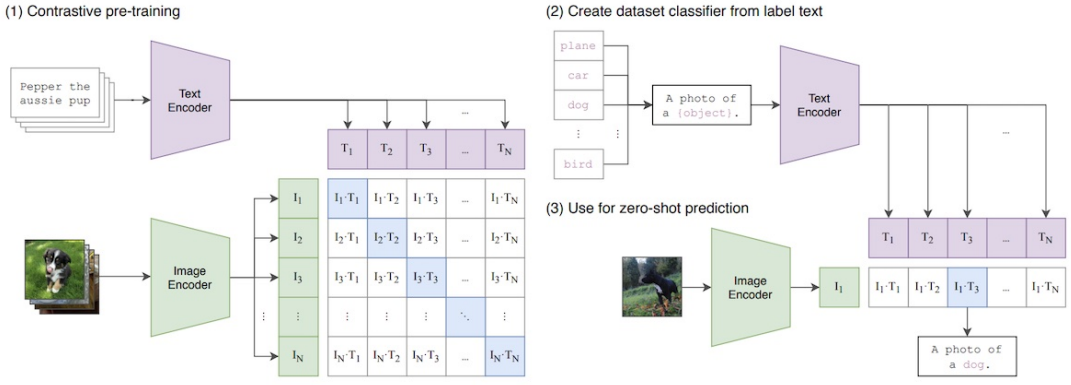



*Figure 1.* Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

# CLIP-guided diffusion (UnClip, DALL-E2)

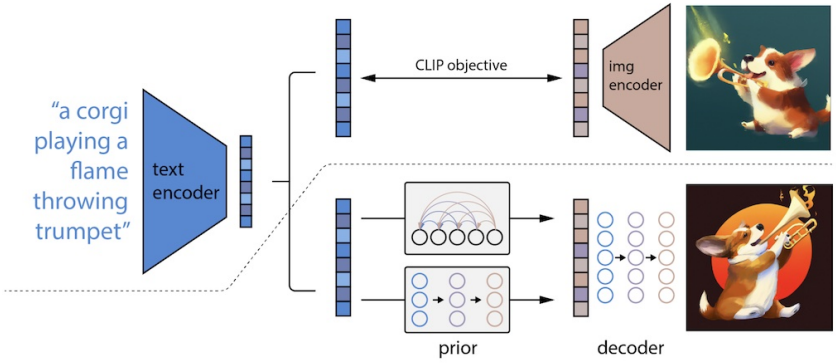



Figure 2: A high-level overview of unCLIP. Above the dotted line, we depict the CLIP training process, through which we learn a joint representation space for text and images. Below the dotted line, we depict our text-to-image generation process: a CLIP text embedding is first fed to an autoregressive or diffusion prior to produce an image embedding, and then this embedding is used to condition a diffusion decoder which produces a final image. Note that the CLIP model is frozen during training of the prior and decoder.

# DALL-E2. Autoregressive

**Autoregressive Approach**

- Similar to classic DALL · E: text conditioning is achieved by placing the text embedding **early in the sequence**.
- A special **dot product token** (<SEP>) is prepended between the text and image embeddings:

$$[e_t^1, \ldots, e_t^n, \langle \text{SEP} \rangle, e_i^1, \ldots, e_i^m]$$

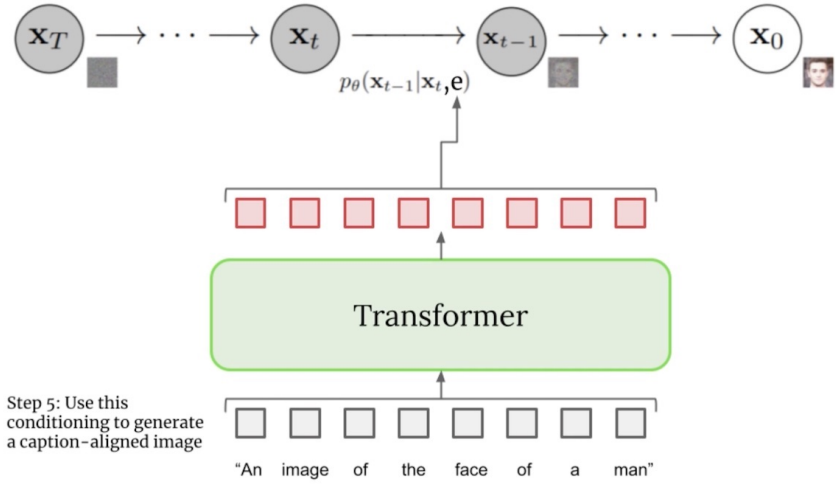
# DALL-E2. Diffusion Approach

- A **decoder-only transformer** with a **causal attention mask** is trained on a concatenated sequence:
  - Encoded text
  - Text embedding
  - Time step embedding
  - Noised CLIP image embedding
  - Final placeholder embedding
- The model predicts the **unnoised CLIP image embedding** from the final token.
- Unlike DDPM, it directly predicts the denoised image embedding:

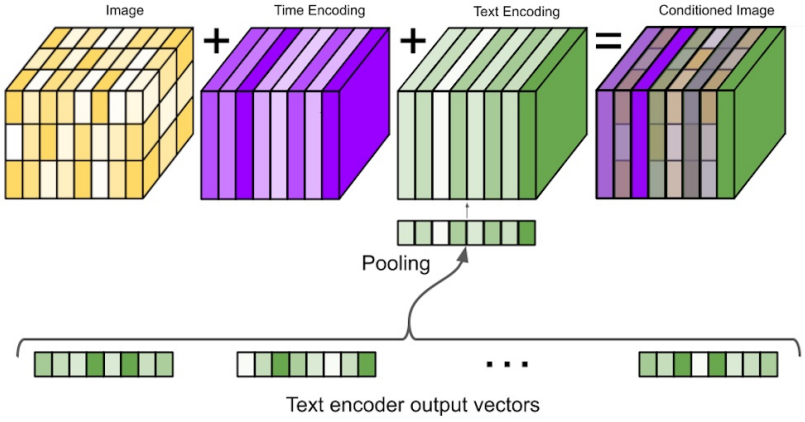
$$\hat{x}_0 = f(x_t, t, e_t)$$

instead of predicting the noise $\epsilon$ and subtracting it.

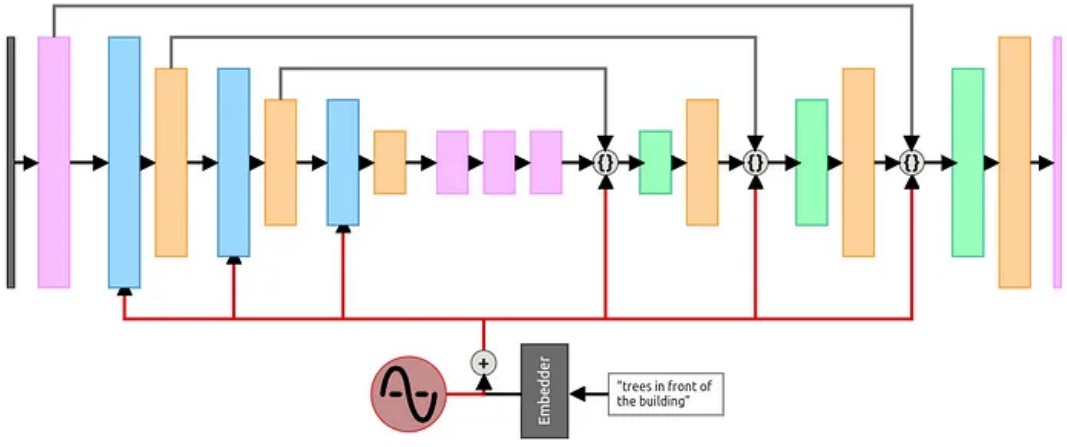
# Imagen


$\mathbf{x}_T$
$\mathbf{x}_t$
$\mathbf{x}_{t-1}$
$\mathbf{x}_0$
$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{e})$
Transformer
Step 5: Use this conditioning to generate a caption-aligned image
"An image of the face of a man"

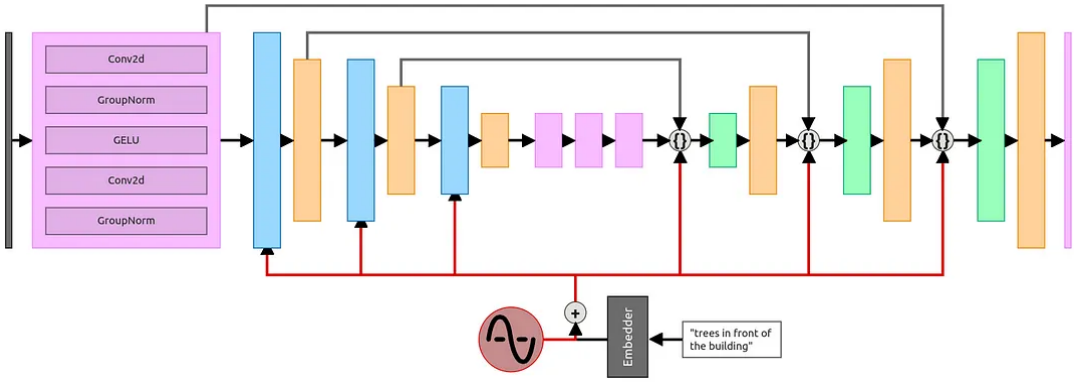
# Imagen. Text embedding



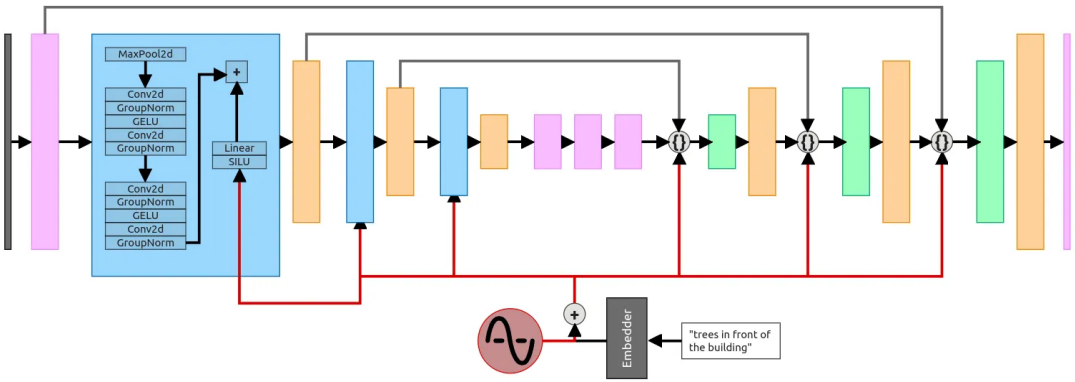
# Imagen. Unet Architecture



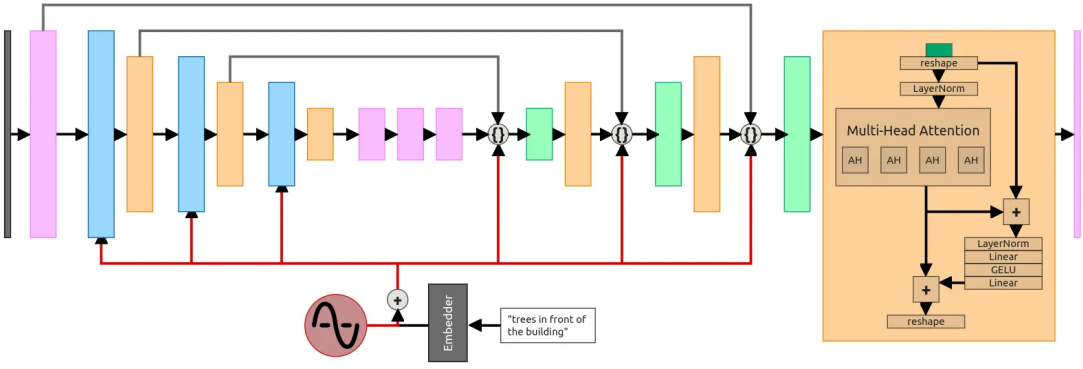
# Imagen. ResNet block



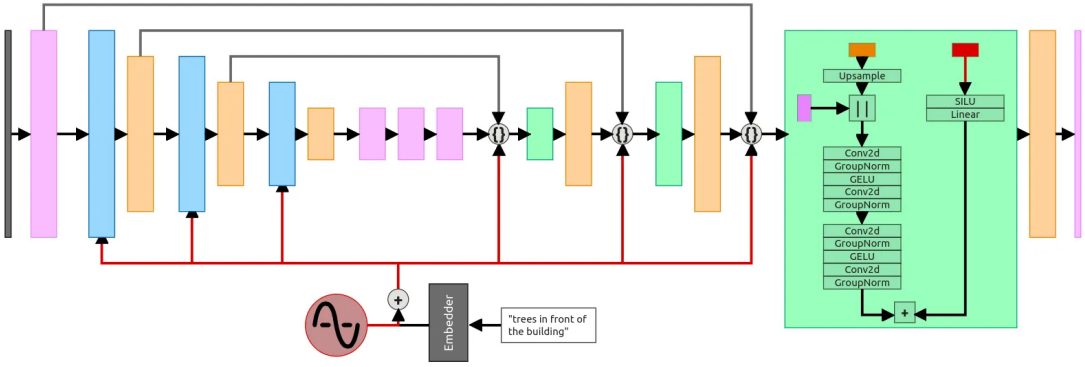
# Imagen. Downsample block



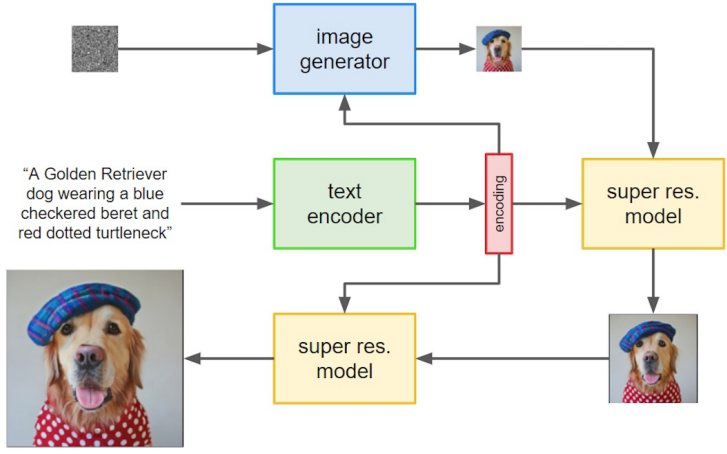
# Imagen. Self-attention block



# Imagen. Upsample block

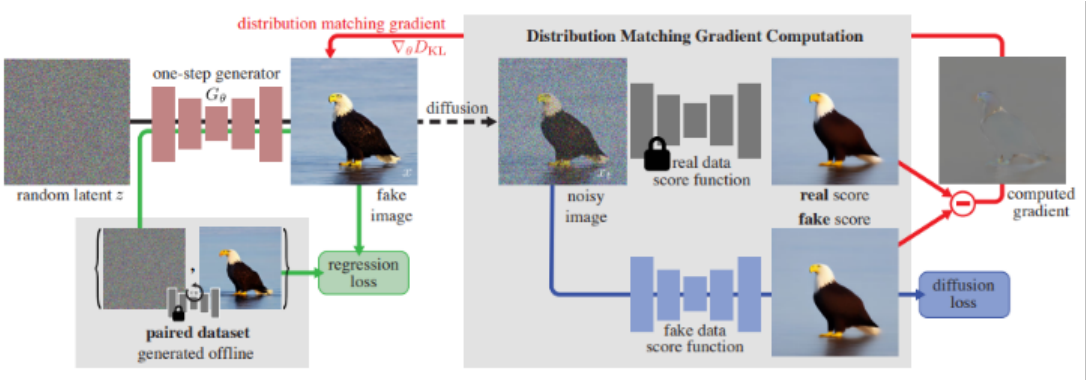


# Imagen. High res



# Diffusion speedup

# One-step Diffusion with Distribution Matching Distillation




https://arxiv.org/pdf/2311.18828

# One-step Diffusion with Distribution Matching Distillation

$$D_{KL}(p_{\text{fake}} \parallel p_{\text{real}}) = \mathbb{E}_{x\sim p_{\text{fake}}}\left(\log \frac{p_{\text{fake}}(x)}{p_{\text{real}}(x)}\right) = \mathop{\mathbb{E}_{z\sim\mathcal{N}(0;I)}}_{x=G_\theta(z)}\left[-\left(\log p_{\text{real}}(x) - \log p_{\text{fake}}(x)\right)\right] \tag{7}$$

$$\nabla_\theta D_{KL} = \mathop{\mathbb{E}_{z\sim\mathcal{N}(0;I)}}_{x=G_\theta(z)}\left[-\left(s_{\text{real}}(x) - s_{\text{fake}}(x)\right)\frac{dG}{d\theta}\right] \tag{8}$$

$$\text{where } s_{\text{real}}(x) = \nabla_x \log p_{\text{real}}(x), \quad s_{\text{fake}}(x) = \nabla_x \log p_{\text{fake}}(x) \tag{9}$$

# One-step Diffusion with Distribution Matching Distillation. Algorithm

**Algorithm 1:** DMD Training procedure

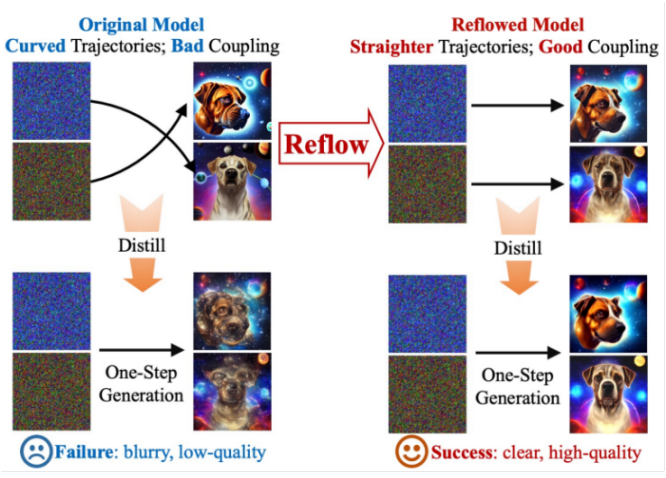
**Input:** Pretrained real diffusion model $\mu_{\text{real}}$, paired dataset $\mathcal{D} = \{z_{\text{ref}}, y_{\text{ref}}\}$
**Output:** Trained generator $G$.

// Initialize generator and fake score estimators from pretrained model
$G \leftarrow \text{copyWeights}(\mu_{\text{real}})$, $\mu_{\text{fake}} \leftarrow \text{copyWeights}(\mu_{\text{real}})$
**while** *train* **do**
  // Generate images
  Sample batch $z \sim \mathcal{N}(0, \mathbf{I})^B$ and $(z_{\text{ref}}, y_{\text{ref}}) \sim \mathcal{D}$
  $x \leftarrow G(z)$, $x_{\text{ref}} \leftarrow G(z_{\text{ref}})$
  $x = \text{concat}(x, x_{\text{ref}})$ **if** dataset is LAION **else** $x$

  // Update generator
  $\mathcal{L}_{\text{KL}} \leftarrow \text{distributionMatchingLoss}(\mu_{\text{real}}, \mu_{\text{fake}}, x)$ // Eq 7
  $\mathcal{L}_{\text{reg}} \leftarrow \text{LPIPS}(x_{\text{ref}}, y_{\text{ref}})$ // Eq 9
  $\mathcal{L}_G \leftarrow \mathcal{L}_{\text{KL}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}$
  $G \leftarrow \text{update}(G, \mathcal{L}_G)$

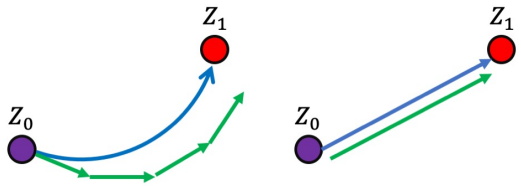
  // Update fake score estimation model
  Sample time step $t \sim \mathcal{U}(0, 1)$
  $x_t \leftarrow \text{forwardDiffusion}(\text{stopgrad}(x), t)$
  $\mathcal{L}_{\text{denoise}} \leftarrow \text{denoisingLoss}(\mu_{\text{fake}}(x_t, t), \text{stopgrad}(x))$ // Eq 6
  $\mu_{\text{fake}} \leftarrow \text{update}(\mu_{\text{fake}}, \mathcal{L}_{\text{denoise}})$
**end while**

# InstaFlow




https://arxiv.org/abs/2309.06380

# InstaFlow. Ode curve

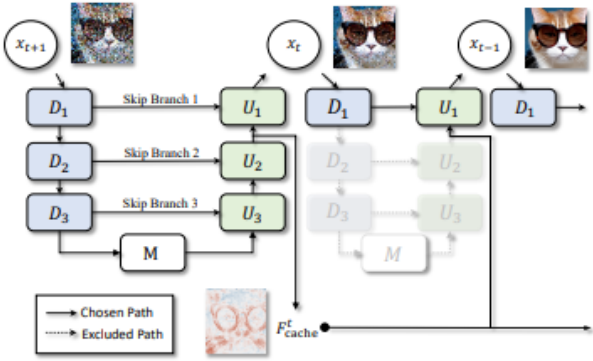



$$v_{k+1} = \arg\min_{v} \mathbb{E}_{X_0 \sim \pi_0, \mathcal{T} \sim D_{\mathcal{T}}} \left[ \int_0^1 \| (X_1 - X_0) - v(X_t, t \mid \mathcal{T}) \|^2 \, dt \right] \tag{10}$$

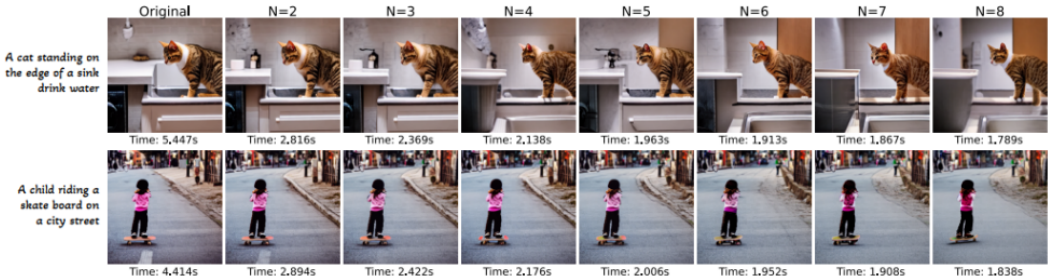
with

$$X_1 = \mathrm{ODE}[v_k](X_0 \mid \mathcal{T}) \quad \text{and} \quad X_t = tX_1 + (1-t)X_0 \tag{11}$$

# DeepCache


$x_{t+1}$
$x_t$
$x_{t-1}$
$D_1$
$D_2$
$D_3$
M
$U_1$
$U_2$
$U_3$
Skip Branch 1
Skip Branch 2
Skip Branch 3
Chosen Path
Excluded Path
$F^t_{\text{cache}}$

# DeepCache. Results



# Thank you for your attention!

Artem Ryzhikov aryzhikov@hse.ru